ATHANASIOS GOURDOMICHALIS

Numerical Analysis Python Programs

Monte Carlo Numerical Integration of $f(x) = \sqrt{x}$

This document provides:

- A short intro to Monte Carlo Integration
- The program's problem statement (assignment prompt and requirements)
- The code description and implementation details.

1. Introduction to Monte Carlo Integration

- Stochastic Approximation: The **Monte Carlo method** estimates the **definite integral** of a **function** $f(x)$ over an **interval** $[a, b]$ by **uniform random sampling** within a **bounding box** $[a, b] \times [y_{min}, y_{max}]$. The ratio of **sample points** that **fall beneath the curve** to the **total number of points approximates** the area **under the curve** relative to **the box**:

$$I \approx \text{Area (Box)} \times \frac{N_{inside}}{N_{total}}$$

- Convergence and Sample Size: **By the Law of Large Numbers** and the **Central Limit Theorem**, the **statistical error scales** as $O(1/\sqrt{n})$, meaning that **increasing** the **sample size n** leads to **a more precise approximation** and **a monotonic reduction** in variance.

2. Problem Statement

- Integral to Approximate:

$$I = \int_0^1 \sqrt{x}\, dx$$

- Sampling Domain: The unit square $[0, 1] \times [0, 1]$.
  For any $x \in [0, 1]$, the curve $f(x) = \sqrt{x}$ satisfies $0 \leq f(x) \leq 1$, so the domain **completely bounds the region of integration**.

- **Requirements:**

1. **Simulation:** We **approximate** the **integral** using the **Monte Carlo method** for **three distinct sample sizes**:

$$n_1 = 1000, \quad n_2 = 4000, \quad n_3 = 16000$$

The area is **estimated** as **the fraction** of **points** falling in the **region** $0 \le p_y \le \sqrt{p_x}$.

2. **Dedicated Plotting Function (`mygraph(…)`):**
We **construct** and **call** a **function** `mygraph(n_i, x_in, y_in, x_out, y_out, estimate)` **three times** (**once per sample size**). The **plots** must **show** the curve $f(x) = \sqrt{x}$, points **inside the region** marked **in blue circles** ('bo'), and **points outside** marked **in red asterisks** ('r*'), complete with **titles**, **labels**, **grids**, and **legends**.

3. **Analytical Verification with SymPy:**
We evaluate **the exact integral** using `sympy.integrate` and **compute** the **absolute numerical error** for **each sample size** to **verify convergence**.

3. Code Description & Implementation Details

- **Initial Setup & Helper Functions:**
  - We import `numpy`, `matplotlib.pyplot`, and `sympy` as `sp`.
  - **Reproducibility** is established using `np.random.seed(0)`.
  - `random(a, b)` generates **uniform floating-point coordinates** over $[a, b]$ via $a + (b - a) \times$ `rand()`.
  - Integrand $f(x) = \sqrt{x}$ is declared **using** `np.sqrt(x)`.
- **The Plotting Function (`mygraph()`):**
  - Receives the **sample size** n_i, **coordinates** of **accepted/rejected points**, and the **computed estimate**.

- Renders the **classified scatter coordinates alongside** the **continuous line** $f(x) = \sqrt{x}$ evaluated **on 300 grid points** via `np.linspace(0, 1, 300)`.
- **Axis limits** are locked to $[0, 1] \times [0, 1]$ with an **equal aspect ratio**.

- **Simulation Execution:**
  - We **iterate over** `n_values = [1000, 4000, 16000]`.
  - In **each trial**, counters `tot` and `cntIn` are **initialized** along with **coordinate storage lists**.
  - **Sample points** (p_x, p_y) are **generated** in $[0, 1] \times [0, 1]$.
  - **Acceptance Criterion:** If $p_y \leq f(p_x)$, the **point lies in the region**; `cntIn` increments, and the **coordinates append to** $(x_{\text{in}}[\,], y_{\text{in}}[\,])$. Otherwise, **they append to** $(x_{\text{out}}[\,], y_{\text{out}}[\,])$.
  - The **integral estimate** is **calculated** as `cntIn/tot` and **plotted** by **invoking** `mygraph()`.

- **Analytical Benchmark & Convergence Analysis:**
  - **Evaluating via SymPy:**

$$I_{\text{exact}} = \int_0^1 \sqrt{x}\, dx = \left[\frac{2}{3}x^{3/2}\right]_0^1 = \frac{2}{3} \approx 0.6667$$

- **Experimental results demonstrate high accuracy across all runs:**
  - $n_1 = 1000 \Rightarrow$ `Estimate` $= 0.6600$ (`Error` $= 0.0067$)
  - $n_2 = 4000 \Rightarrow$ `Estimate` $= 0.6562$ (`Error` $= 0.0104$)
  - $n_3 = 16000 \Rightarrow$ `Estimate` $= 0.6596$ (`Error` $= 0.0071$)